

Projekt 5 — Uczenie przez wzmacnianie w przestrzeniach ciągłych

Autonomiczny agent wyścigowy (SAC + CarRacing-v3)

Autorzy

	Autor 1	Autor 2
Imię i nazwisko	Patryk Chamera	Karol Bystrek
E-mail	pchamera@student.agh.edu.pl	karbystrek@student.agh.edu.pl

Przedmiot: Inteligencja obliczeniowa

Temat: Reinforcement learning w ciągłej przestrzeni akcji i obserwacji

Repozytorium: racing-agent-gymnasium

Biblioteka RL: Stable-Baselines3

Środowisko: Gymnasium CarRacing-v3

1. Cel projektu

Celem projektu było zaimplementowanie i wytrenowanie agenta uczącego się przez wzmacnianie w **ciągłej** przestrzeni akcji, z obserwacją wizualną (obraz), oraz przygotowanie raportu obejmującego: opis środowiska i algorytmu, porównanie zestawów hiperparametrów, porównanie architektur sieci CNN, pomiar czasu uczenia oraz ewaluację deterministyczną najlepszego agenta.

Projekt jest kontynuacją wcześniejszej pracy w **przestrzeniach dyskretnych** (crossy-road-gymnasium, PPO). Tym razem zastosowaliśmy algorytm **off-policy** (SAC) oraz politykę **CnnPolicy** ze względu na obserwacje pikselowe.

2. Wybrane środowisko: CarRacing-v3

2.1 Opis

CarRacing-v3 (Gymnasium / Box2D) to klasyczne środowisko wyścigowe z widokiem z góry. Tor jest losowany w każdym epizodzie, co wymusza generalizację, a nie zapamiętanie jednej trasy.

Przestrzeń obserwacji: `Box(0, 255, (96, 96, 3), uint8)` — kolorowy obraz RGB 96×96 px.

Przestrzeń akcji (ciągła): `Box([-1, 0, 0], [1, 1, 1], (3,), float32)` :

- oś 0 — kierownica (−1 = maks. w lewo, +1 = maks. w prawo),
- oś 1 — gaz,
- oś 2 — hamulec.

Funkcja nagrody: w każdej klatce agent otrzymuje ok. −0,1 (kara za czas) oraz dodatni składnik za **nowo odwiedzone fragmenty toru**; powtórne najechanie na te same kafelki daje silną karę. Nagroda jest więc gęsta, ale wymaga utrzymania się na torze i pokonywania łuków.

Koniec epizodu: ukończenie okrążenia lub limit 1000 kroków.

2.2 Preprocessing obserwacji (implementacja)

W repozytorium (`src/racing_agent/env/wrappers.py` , `make_env.py`) stosujemy kolejno:

1. **Skala szarości** — $(96, 96, 3) \rightarrow (96, 96)$.
2. **Resize** — domyślnie 84×84 (lokalnie) lub 64×64 (profil Kaggle).
3. **Frame stack** — domyślnie 4 klatki (lokalnie) lub 2 klatki (profil Kaggle użyty w treningu).
4. Opcjonalnie **ClipReward** (wyłączone w naszych runach).

Wyjście do `CnnPolicy` ma postać **C×H×W** (uint8), zgodnie z konwencją SB3.

2.3 Czas kroku i epizodu

Na podstawie runów @ 100 000 kroków (GPU T4, Kaggle, `arch_light_cnn`):

Metryka	Wartość
Średni czas kroku	~0,025 s

Metryka	Wartość
Średni czas epizodu	~1263 s (~21 min) — epizody treningowe liczone przez Monitor SB3
Średni czas ściany (wall-clock) / 100k kroków	~2529 s (~42 min)

Uwaga: czas epizodu w Monitorze obejmuje pełne interakcje treningowe (w tym aktualizacje SAC), dlatego jest dłuższy niż sam rollout w podglądzie.

3. Wybrany algorytm: Soft Actor-Critic (SAC)

3.1 Opis

Soft Actor-Critic (SAC) to algorytm **off-policy** dla ciągłych akcji, łączący:

- **aktor-krytyk** z funkcją Q (w SAC: **dwie** sieci Q — double Q, redukcja przeszacowania),
- **maksymalizację entropii** — polityka maksymalizuje nagrodę i jednocześnie utrzymuje eksplorację (temperatura α),
- **bufor doświadczeń (replay buffer)** — ponowne wykorzystanie próbek, istotne przy drogich obserwacjach wizualnych.

W **Stable-Baselines3** używamy klasy `SAC` z polityką `CnnPolicy`: osobny ekstraktor cech CNN (NatureCNN / CustomDeepCNN / LightCNN) oraz głowy aktora i krytyków MLP.

Kluczowe cechy konfiguracji SB3 w naszym projekcie:

- `ent_coef="auto"` — automatyczna regulacja temperatury,
- `train_freq`, `gradient_steps`, `batch_size`, `tau` — sterowanie tempem uczenia i stabilnością,
- zapis `best_model.zip` przez callback ewaluacyjny (`EvalSaveBestCallback`).

Jak to działa — analogia kierowcy

Cały agent można potraktować jak **kierowcę autonomicznego**, złożonego z kilku sieci neuronowych:

Rola w analogii	Element w SAC	Co robi
Oczy	CNN (np. LightCNN)	Patrzy na obraz z gry (klatki toru + samochód), wyciąga z pikseli wektor cech — np. 128 liczb opisujących sytuację na torze
Decyzja	Aktor (MLP)	Na podstawie cech wybiera akcję: kierownica, gaz, hamulec (3 liczby ciągłe)
Ocena	Krytyk (2× MLP, Q1 i Q2)	Ocenia parę (<i>stan</i> , <i>akcja</i>): „czy ta decyzja w tej sytuacji da dobrą nagrodę w dłuższej perspektywie?”
Trener	Algorytm SAC	Porównuje decyzje z nagrodami ze środowiska i poprawia wagi CNN, aktora i krytyków

Przepływ danych podczas treningu:

1. Środowisko CarRacing-v3 zwraca **obraz** (po preprocessingu: skala szarości, resize, frame stack).
2. CNN zamienia obraz na **wektor cech**.
3. **Aktor** na tym wektorze wybiera akcję i wysyła ją do symulatora.
4. Gra zwraca **nagrodę** (np. kara $-0,1$ za czas, bonus za nowe fragmenty toru) oraz **następny obraz**.
5. Para (*obraz*, *akcja*, *nagroda*, *następny obraz*) trafia do **bufora doświadczeń**.
6. SAC losuje batch z bufora i aktualizuje wagi wszystkich sieci (backprop).

Ważne: nagrody i kary **nie są „w środku” sieci** — nadaje je symulator gry. Sieci używają ich wyłącznie jako sygnału do uczenia. Przy podglądzie (`watch_agent.py`) działa tylko ścieżka **obraz** → CNN → aktor → akcja; krytyk nie steruje samochodem.

Skład „Soft” w nazwie oznacza **maksymalizację entropii**: oprócz wysokiej nagrody SAC nagradza też pewną losowość akcji, żeby agent nie utknął w jednej, złej strategii. Parametr α (`ent_coef="auto"` w SB3) balansuje między zbiorem nagrody a eksploracją.

3.2 Dlaczego SAC, a nie PPO?

W Projekt 4 (Crossy Road) używaliśmy **PPO** (on-policy) na wektorowej obserwacji. Tutaj obserwacja to **obraz**, a rollout jest kosztowny — SAC z buforem doświadczeń lepiej wykorzystuje zebrane dane. Ciągła akcja (sterowanie + gaz + hamulec) jest naturalnym zastosowaniem SAC w SB3.

4. Implementacja w repozytorium

Moduł	Ścieżka	Rola
Fabryka środowiska	src/racing_agent/env/make_env.py	make_car_racing() , make_car_racing_single()
Wrappery wizualne	src/racing_agent/env/wrappers.py	grayscale, resize, frame stack
Polityki CNN	src/racing_agent/policies/	NatureCNN , CustomDeepCNN , LightCNN
Trening	src/racing_agent/training/train.py	klasa Trainer , SAC, callbacks
Hiperparametry	configs/hp_*.yaml , configs/arch_*.yaml	YAML → merge → SAC
Eksperymenty	scripts/run_experiment.py	siatka HP × seedy
Krzywe uczenia	scripts/plot_curves.py	średnia ± std, timing_table.csv
Podgląd na żywo	scripts/watch_agent.py	pygame, render_mode="human"
Ewaluacja	scripts/evaluate.py	50 epizodów, deterministic=True

Każdy run zapisuje metadane w `experiments/<run_id>/run_metadata.json` , logi Monitor w `logs/monitor/` , modele w `models/best/` i `models/final/` .

Trening produkcyjny wykonywaliśmy na **Kaggle (GPU T4)** z notebookiem `notebooks/02_kaggle_hp_sweep.ipynb` ; wyniki importowaliśmy lokalnie.

5. Zestawy hiperparametrów

Zgodnie z poleceniem zdefiniowaliśmy **trzy** zestawy HP (pliki `configs/`). W treningu Kaggle faktycznie wykorzystaliśmy `**hp_baseline**`; pozostałe dwa zestawy są gotowe w repozytorium do uruchomienia siatki (`run_experiment.py`).

5.1 Tabela hiperparametrów SAC

Parametr	hp_baseline	hp_high_lr	hp_large_batch
learning_rate	3×10^{-4}	1×10^{-3}	3×10^{-4}
buffer_size	100 000	100 000	300 000
batch_size	256	256	512
tau	0,005	0,02	0,005
train_freq	1	1	4
gradient_steps	1	1	4
learning_starts	1000	1000	2000
gamma	0,99	0,99	0,99
ent_coef	auto	auto	auto

Hipotezy:

- *baseline* — punkt odniesienia (domyślne ustawienia SB3),
- *high_lr* — szybsze uczenie, ryzyko niestabilności,
- *large_batch* — gładzsze gradienty, wyższy koszt czasu ściany.

5.2 Profil Kaggle (kaggle_overrides.yaml)

Ze względu na limit czasu GPU zastosowaliśmy dodatkowy plik overrides:

Parametr	Wartość w treningu	Cel
resize_to	64	mniej pikseli → szybszy krok
frame_stack	2	mniejsze wejście CNN
buffer_size	50 000	szybszy start uczenia
learning_starts	500	wcześniejsze update'y
eval_every_timesteps	50 000	zapis <code>best_model</code> na końcu runu

Architektura treningowa: `**arch_light_cnn**` (mniejsza sieć niż NatureCNN).

6. Architektury sieci neuronowych

Polecenie wymaga co najmniej **dwóch** architektur CNN. W repozytorium zaimplementowaliśmy trzy ekstraktory cech; w **zakończonych eksperymentach** używaliśmy **LightCNN**. **NatureCNN** i **CustomDeepCNN** są w pełni zaimplementowane i przewidziane do porównania (Phase 5); lokalnie wykonano krótki smoke test NatureCNN (5000 kroków).

6.1 Architektura A — NatureCNN (SB3 domyślna)

- **Wejście:** (4, 84, 84) float32 (po normalizacji /255),
- **Conv** 32×8, stride 4 → ReLU → Conv 64×4, s=2 → ReLU → Conv 64×3, s=1 → ReLU,
- **Flatten** → **Linear** → 512 (features_dim),
- **Głowy SAC:** MLP [256, 256] (aktor i krytyki).

Plik: configs/arch_nature_cnn.yaml , src/racing_agent/policies/nature_cnn.py .

Schemat architektury NatureCNN

NatureCNN (arch_nature_cnn)						
Layer	Kernel	Stride	Output shape	Activation	Params	Notes
Conv2d + ReLU	8×8	4	(32, 20, 20)	ReLU	~21k (in→32 ch)	n_in = 4
Conv2d + ReLU	4×4	2	(64, 9, 9)	ReLU	~33k	
Conv2d + ReLU	3×3	1	(64, 7, 7)	ReLU	~37k	
Flatten + Linear + ReLU	—	—	(features_dim,)	ReLU	~1.6M (3136→512)	features_dim default 512
Σ trainable parameters					1,684,128	

6.2 Architektura B — CustomDeepCNN

- **Wejście:** (4, 84, 84) ,
- **Conv** 32×3, s=2 → BatchNorm → ReLU → (powtarzane warstwy 64, 128, 128),
- **AdaptiveAvgPool2d(1)** → **Linear** → 256,
- **Głowy SAC:** MLP [256, 256] (identyczne jak w A — uczciwe porównanie).

Plik: configs/arch_deep_cnn.yaml , src/racing_agent/policies/custom_cnn.py .

Schemat architektury CustomDeepCNN

CustomDeepCNN (arch_deep_cnn)

Layer	Kernel	Stride	Output shape	Activation	Params	Notes
Conv2d + BN + ReLU	3×3	2	(32, 41, 41)	ReLU	(see total)	in_ch = n_input_channels
Conv2d + BN + ReLU	3×3	2	(64, 20, 20)	ReLU		
Conv2d + BN + ReLU	3×3	2	(128, 9, 9)	ReLU		
Conv2d + BN + ReLU	3×3	1	(128, 7, 7)	ReLU		
AdaptiveAvgPool2d(1)	—	—	(features_dim,)	ReLU		pool → 128-d vector
Σ trainable parameters					274,848	

6.3 Architektura C — LightCNN (użyta w treningu Kaggle)

- Wejście: (2, 64, 64) przy overrides Kaggle,
- Conv 16×8, s=4 → ReLU → Conv 32×4, s=2 → ReLU,
- Flatten → Linear → 128,
- Głowy SAC: MLP [128] .

Plik: configs/arch_light_cnn.yaml — ok. 5–10× mniej parametrów conv niż NatureCNN, ~42 min / 100k kroków na T4.

Schemat architektury LightCNN (użyta w treningu)

LightCNN (arch_light_cnn)

Layer	Kernel	Stride	Output shape	Activation	Params	Notes
Conv2d + ReLU	8×8	4	(16, 15, 15)	ReLU		in_ch = n_input_channels
Conv2d + ReLU	4×4	2	(32, 6, 6)	ReLU		
Flatten + Linear + ReLU	—	—	(features_dim,)	ReLU		default features_dim=128
Σ trainable parameters					157,872	

Uzasadnienie wyboru LightCNN w treningu: pełna siatka 3×10×50k na NatureCNN przekraczałyby budżet Kaggle (~30 h GPU/tydzień); LightCNN umożliwił iterację i dopracowanie pipeline’u importu, podglądu i ewaluacji.

7. Przebieg eksperymentów i etapy uczenia

Poniżej chronologia zakończonych runów (hp_baseline , arch_light_cnn , overrides Kaggle, stack=2, 64×64).

Etap	Run (skrót)	Seed	Kroki	Szczyt nagrody (Monitor)	Obserwacja
1 — smoke Kaggle	...193403	0	20 000	−20	Agent prawie losowy; wirowanie / brak jazdy
1 — smoke Kaggle	...194312 , ...195214	1, 2	20 000	−17, 0,4	Brak sensownej jazdy
2 — dłuższy run	...205005	0	100 000	288	Pierwsze dodatnie epizody; niestabilność
3 — long run	...214604	0	300 000	863	Najwyższy szczyt treningu; końcówka treningu gorsza (średnia ostatnich ep. ujemna)
4 — sweep (sesja 1)	...061846	0	100 000	288	Powtórka seed0 @ 100k (nowszy timestamp)
4 — sweep (sesja 1)	...070055	1	100 000	58	Słaby run — brak stabilnej jazdy
4 — sweep (sesja 1)	...074316	2	100 000	774	Najlepszy run pod kątem jazdy w podglądzie

7.1 Co poprawialiśmy w trakcie projektu

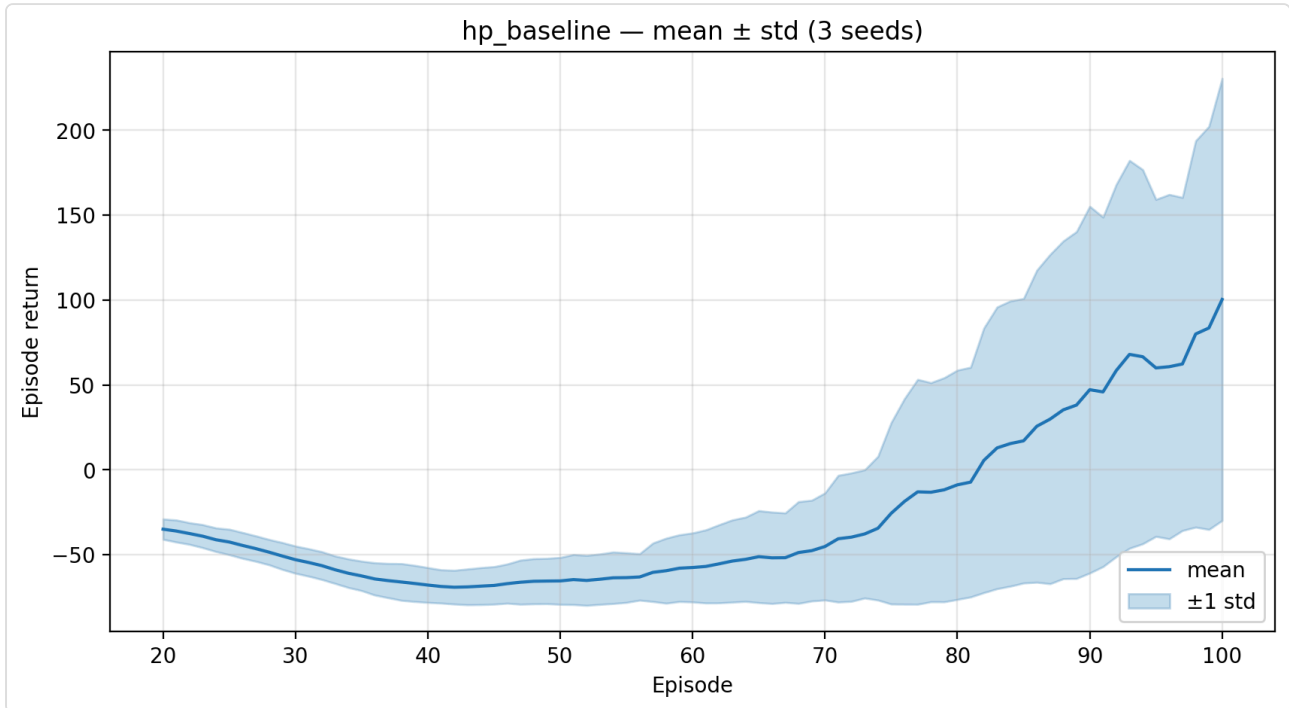
1. Zwiększenie liczby kroków (20k → 100k → 300k) — kluczowa poprawa; przy 20k agent nie miał szansy nauczyć się toru.
2. Wybór checkpointu — porównanie `best_model` vs `final_model` ; auto-wybór po szczytowej nagrodzie Monitor vs jakość jazdy w podglądzie.
3. Ewaluacja deterministyczna — 50 epizodów headless (`evaluate.py`).

8. Wyniki ilościowe

8.1 Krzywa uczenia

Poniżej krzywa uczenia dla ****hp_baseline , architektura arch_light_cnn , 3 seedy @ 100 000 kroków**** (średnia $\pm$ odch. std.):

Krzywa uczenia hp_baseline — mean $\pm$ std (3 seedy)



Duży rozstrzał między seedami (seed1 słaby, seed2 silny) potwierdza wysoką wariancję SAC na CarRacing przy ograniczonej liczbie kroków. Widać poprawę względem wczesnych runów @ 20k (nagrody ujemne) — efekt zwiększenia budżetu kroków opisany w sekcji 7.

8.2 Czas treningu

Plik: reports/figures/timing_table.csv

HP	Liczba runów	Średni wall-clock [s]	Średni czas kroku [s]	Kroki
hp_baseline	3	2529	0,025	100 000

8.3 Ewaluacja deterministyczna

Ewaluacja to test już wytrenowanego agenta: wczytujemy zapisany checkpoint (best_model.zip), puszcamy 50 epizodów w grze **bez dalszego uczenia** i zbieramy statystyki nagrody (średnia, mediana, min, max). Skrypt: scripts/evaluate.py .

Czym różni się tryb deterministyczny?

W SAC **aktor** dla każdego kroku zwraca nie jedną liczbę, lecz **rozkład akcji** — np. „najlepiej skręcić około $-0,4$, z pewnym rozrzutem”. Dotyczy to trzech wyjść: kierownicy, gazu i hamulca.

Tryb	Parametr	Zachowanie agenta
Deterministyczny	deterministic=True	Zawsze wybiera środek rozkładu — tę samą, „najpewniejszą” akcję przy tym samym obrazie
Stochastyczny	deterministic=False	Losuje akcję z rozkładu — ten sam zakręt może dać nieco inną decyzję

Przykład (kierownica): sieć sugeruje skręt w lewo około $-0,4$. W trybie deterministycznym agent **zawsze** wysła $-0,4$; w stochastycznym może wysłać np. $-0,38$, $-0,52$ lub $-0,31$.

Używamy `**deterministic=True`, **żeby zmierzyć** stabilną jakość** polityki, a nie przypadkowy szum eksploracji z treningu.

Agent porównawczy — seed0 @ 300k

Model: hp_baseline__arch_light_cnn__seed00__20260518-214604 / best_model.zip

Plik: reports/figures/eval_summary.json

Ustawienia: 50 epizodów, deterministic=True .

Statystyka	Wartość
Średnia nagroda	185,1
Mediana	191,6
Odch. std.	174,5
Min	$-40,4$
Max	795,8

Agent demo — seed02 @ 100k (lepszy w podglądzie)

Model: hp_baseline__arch_light_cnn__seed02__20260519-074316 / best_model.zip

Plik: reports/figures/eval_summary_seed02.json

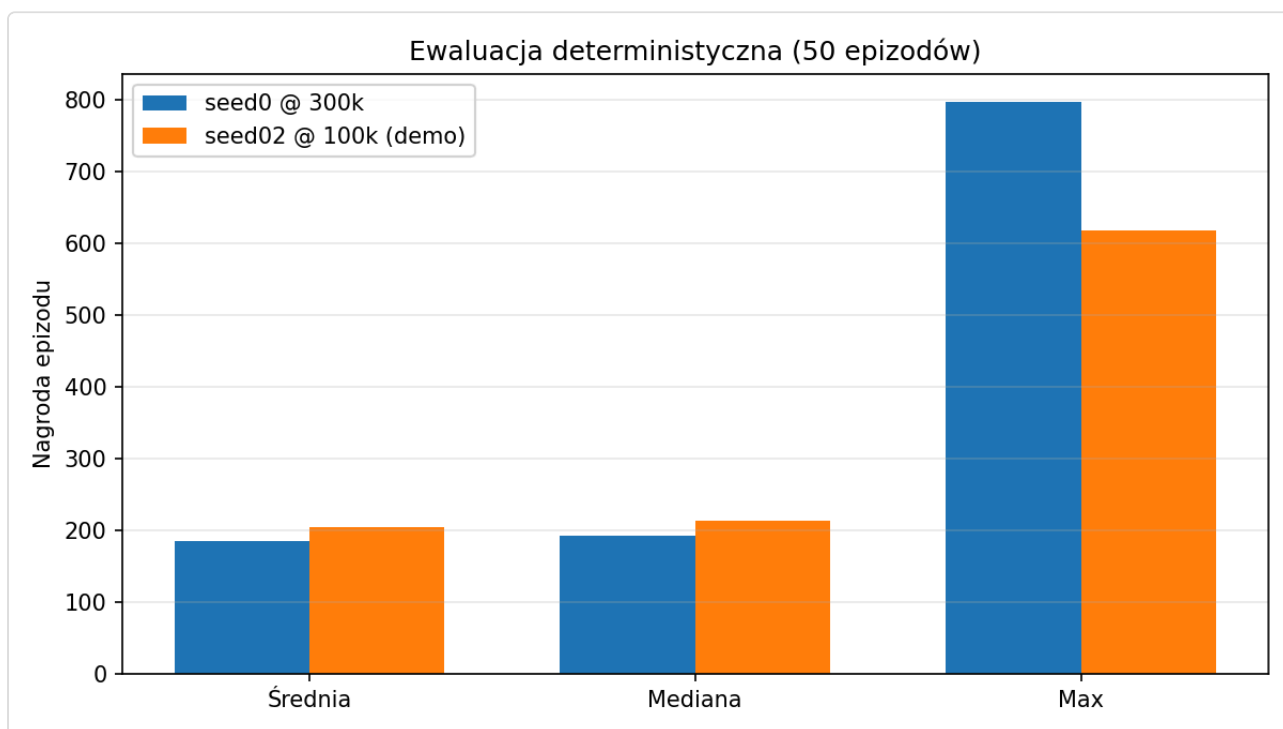
Ustawienia: 50 epizodów, deterministic=True .

Statystyka	Wartość
Średnia nagroda	203,8
Mediana	213,7
Odch. std.	170,9
Min	-52,4
Max	617,5

We wszystkich epizodach ewaluacji długość = **1000 kroków** (timeout). seed02 ma **wyższą średnią i medianę** niż seed0 @ 300k, co potwierdza wybór seed02 jako agenta demo mimo niższego peaku treningowego (774 vs 863).

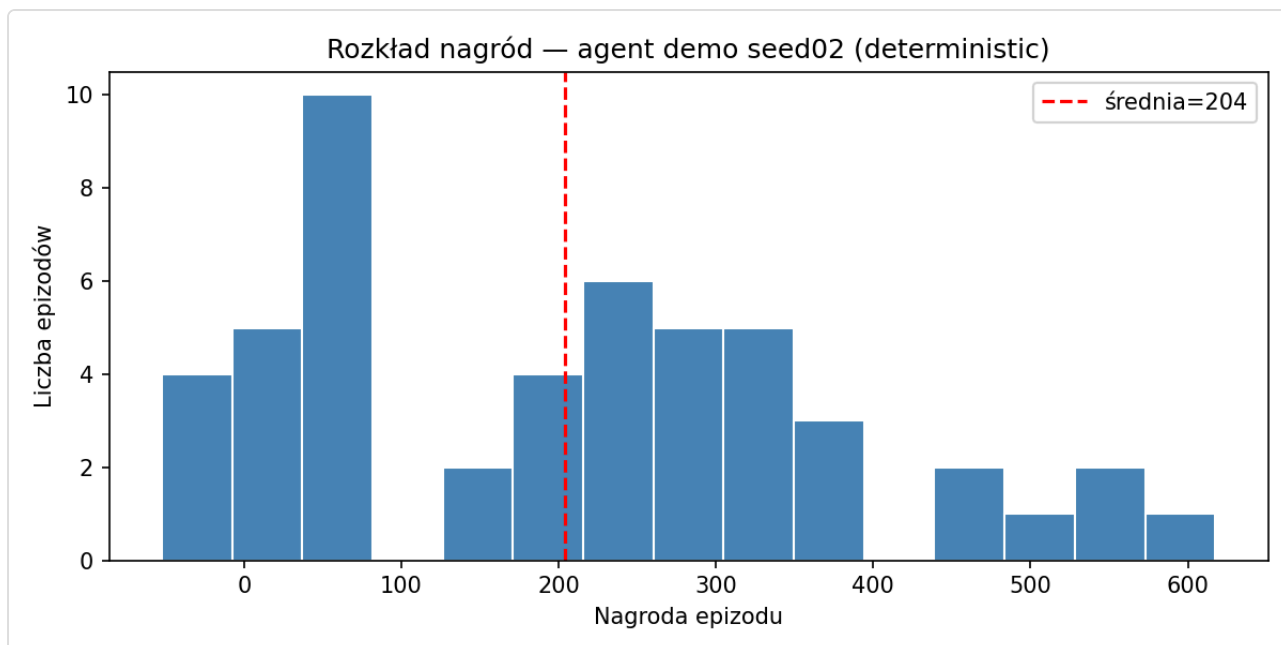
Porównanie obu agentów (ewaluacja deterministyczna, 50 epizodów):

Porównanie ewaluacji deterministycznej seed0 vs seed02



Rozkład nagród — agent demo seed02:

Histogram nagród ewaluacji — seed02 @ 100k



Wysokie odchylenie standardowe (~ 171) wskazuje na dużą zmienność epizodów — część rund kończy się słabo (ujemna nagroda), część bardzo dobrze (ponad 600 pkt).

8.4 Porównanie dwóch agentów demo (podgląd)

Agent	Folder	Peak treningu	Zachowanie w watch_agent
Główny (lepszy)	...seed02__20260519-074316	774	Lepsza jazda na prostych; na ostrym łuku leci prosto poza mapę
Drugie (długi run)	...seed00__20260518-214604	863	Na prostych OK; na łuku hamuje i stoi do końca epizodu ($-0,1/\text{krok}$)

Przykładowe logi podglądu (--fast):

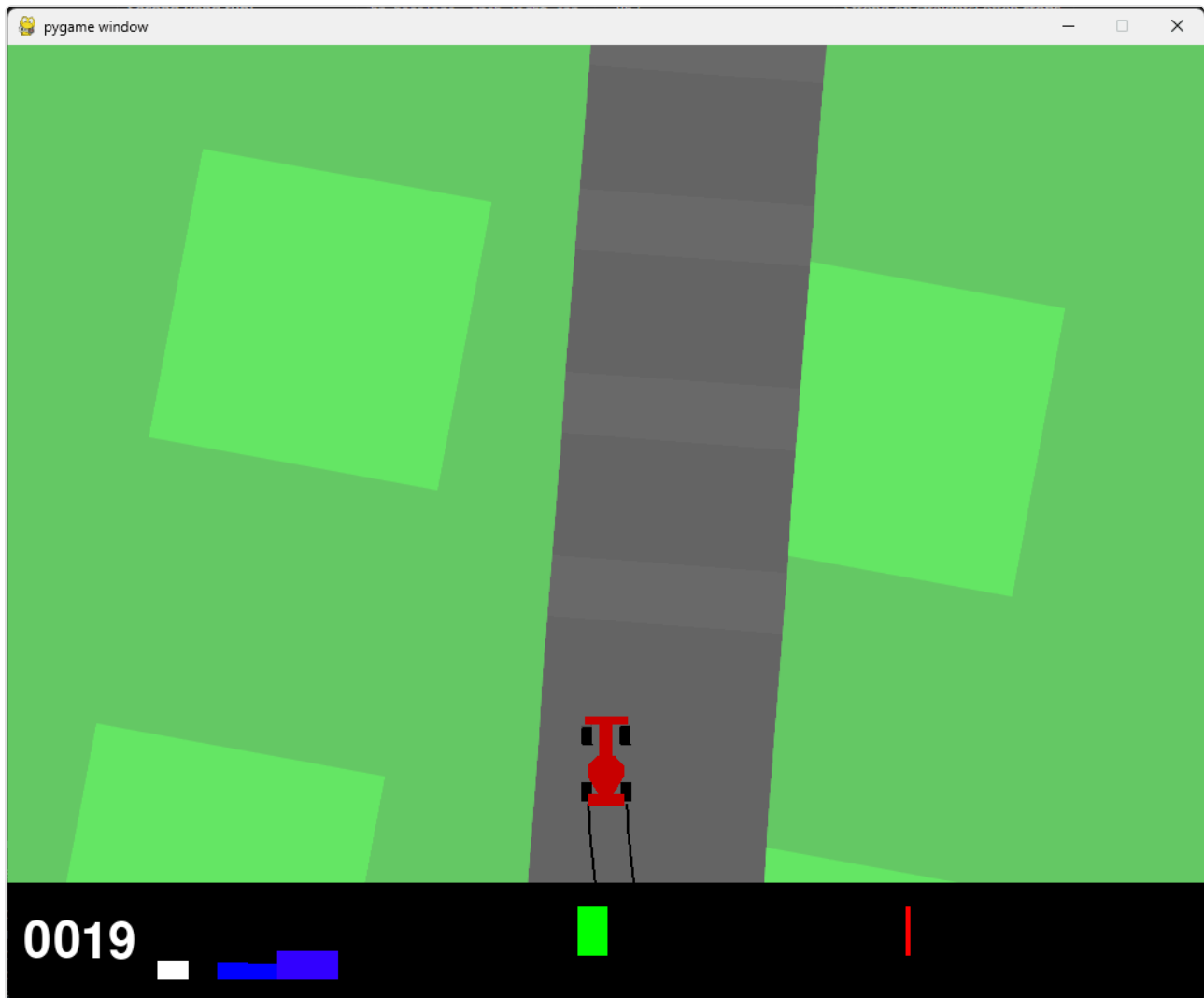
- **seed0 @ 300k:** ep.2 — nagroda 347,8 @ krok 400, potem spadek ~ 20 co 200 kroków (postój).
- **seed02 @ 100k:** ep.2 — nagroda 389,5 @ krok 400 (wyraźnie lepszy fragment jazdy).

Wnioski jakościowe: oba modele nie opanowały ostrych zakrętów — różne „awarie”: postój vs wypadnięcie z toru. seed02 wybraliśmy jako **agent demo** ze względu na lepsze odcinki jazdy w podglądzie mimo nieco niższego peaku treningowego niż seed0 @ 300k.

9. Screenshoty z wyścigu

Poniższe kadry pochodzą z podglądu na żywo (`scripts/watch_agent.py` , agent demo `seed02 @ 100k`, `deterministic=True`). Pokazują typowe zachowanie modelu: poprawną jazdę na prostych odcinkach oraz awarię na ostrym łuku.

9.1 Poprawna jazda — prosty odcinek



Rys. 9.1 — Pojazd jedzie środkiem toru; widoczne ślady opon. Agent utrzymuje kierunek i zbiera nagrodę za postęp po torze.

9.2 Wejście w zakręt



Rys. 9.2 — Pojazd skręca na łuku

9.3 Awaria — wyjazd poza tor



Rys. 9.3 — Pojazd całkowicie opuścił asfalt. Epizod de facto stracony: brak nagrody za trawę, agent nie wraca na tor.

9.4 Dlaczego model nie jest „najlepszy”?

Obecny checkpoint nie reprezentuje pełnego potencjału SAC na CarRacing — wynika to głównie z **ograniczonego budżetu treningu** (100 000 kroków na Kaggle T4, profil LightCNN 64×64, stack=2) oraz braku czasu na pełną siatkę HP i porównanie architektur. Literatura i typowe benchmarki wymagają **milionów kroków** i większej sieci.

Dlaczego przy ostrych zakrętach wypada z toru i nie wraca?

1. **Niedotrenowanie na trudnych manewrach** — przy 100k kroków agent najczęściej widział proste odcinki i łagodne łuki; na ostrym zakręcie trzeba jednocześnie hamować, skręcać i trzymać linię — tej synergii sieć jeszcze nie opanowała.
2. **Zbyt duża prędkość w wejściu w łuk** — polityka często utrzymuje gaz; na prostej to działa, na ostrym łuku bezwładność wypycha auto na zewnętrzną krawędź.
3. **Brak nagrody poza torem** — po wyjeździe na trawę nagroda spada; obraz (zielone tło bez wyraźnej krawędzi toru) **nie przypomina stanów z treningu**, więc aktor nie ma „mapy powrotu”.

4. **Polityka deterministyczna** — w podglądzie agent nie eksploruje; powtarza te same (błędne) akcje zamiast szukać toru.

To zgodne z obserwacjami z sekcji 8.4: seed02 jeździ lepiej niż seed0 na prostych, ale **oba modele nie opanowały ostrych łuków** — różnią się tylko typem awarii (wypadnięcie vs postój).

10. Zadanie 8-punktowe — podsumowanie

Element	Realizacja
Zapis najlepszego agenta	experiments/.../models/best/best_model.zip
Symulacja deterministyczna	scripts/evaluate.py --deterministic
Porównanie z krzywą uczenia	peak treningu (863) vs średnia eval (185) — patrz sekcja 8.1 i wykres powyżej
Podgląd wizualny	scripts/watch_agent.py (pygame)

11. Wnioski końcowe

1. **SAC + CnnPolicy** na CarRacing-v3 jest wykonalny, ale wymaga **dużej liczby kroków** ($\geq 100k$) i stabilnego środowiska treningowego.
2. **Profil Kaggle (LightCNN, 64×64, stack=2)** umożliwił iterację w rozsądnym czasie (~42 min / 100k na T4).
3. **Najlepszy agent demo:** hp_baseline , seed 2, 100k kroków — sensowna jazda na prostych, problemy na ostrych łukach.
4. **Agent porównawczy:** seed 0, 300k — najwyższy peak treningu (863), gorsze zachowanie deterministyczne na łukach (postój).
5. **Hiperparametry i architektury B/C** — zdefiniowane i zaimplementowane; pełna siatka eksperymentów nie została ukończona z powodu limitów obliczeniowych (ujawnione w raporcie zgodnie z wymogiem uzasadnienia).

12. Uruchamianie

```
# środowisko
py -3.12 -m venv .venv && .venv\Scripts\Activate.ps1
pip install -e ".[dev]"

# podgląd – najlepszy agent
python scripts/watch_agent.py \
    --run-dir experiments/hp_baseline__arch_light_cnn__seed02__20260519-074316
\
    --fast --episodes 5

# podgląd – agent porównawczy (300k)
python scripts/watch_agent.py \
    --run-dir experiments/hp_baseline__arch_light_cnn__seed00__20260518-214604
\
    --fast --episodes 5

# ewaluacja deterministyczna
python scripts/evaluate.py \
    --run-dir experiments/hp_baseline__arch_light_cnn__seed00__20260518-214604
\
    --episodes 50 --deterministic

# wykresy
python scripts/plot_curves.py --arch arch_light_cnn --min-timesteps 100000
python scripts/generate_report_figures.py
```